QCS

Research Technical Document v1.0

Quantum Cryptanalysis Suite
Simulated Qubits Doing Real Work

Exact state-vector simulation of QAOA, Shor's algorithm, and Grover's search applied to toy cryptographic instances on gate-level quantum circuit emulation.

ORCID
0009-0008-6017-4922

DOI
10.5281/zenodo.19961317

REPOSITORY
GitHub

PUBLISHED
May 1, 2026

QCS

G

Gerald Enrique Nelson Mc Kenzie

Independent Researcher

Correspondence: github.com/lordxmen2k/Quantum-Cryptanalysis-Suite

Date: May 1, 2026 | **Version:** 1.0 | **License:** Apache 2.0

Abstract

*We present the **Quantum Cryptanalysis Suite (QCS)**, a pedagogical framework implementing exact state-vector simulation of three canonical quantum algorithms applied to toy cryptographic instances. The suite comprises (1) the Quantum Approximate Optimization Algorithm (QAOA) for lattice problems including the Shortest Vector Problem (SVP) and Learning With Errors (LWE), (2) Shor's period-finding algorithm for integer factorization on composite numbers up to N = 35, and (3) Grover's amplitude amplification for symmetric key search on 4–6 bit toy ciphers. All algorithms execute on a lightweight, gate-level state-vector simulator supporting up to ~16 qubits on consumer hardware. We provide rigorous architectural documentation, experimental validation, and explicit mappings from heuristic concepts to their corresponding quantum algorithmic primitives. This work serves as an educational reference for understanding the mechanics of quantum cryptanalysis algorithms through executable simulation.*

Quantum Computing | Cryptanalysis | QAOA | Shor's Algorithm | Grover's Algorithm | Lattice Cryptography | State-Vector Simulation

§01 Introduction

The intersection of quantum computing and cryptanalysis has produced some of the most consequential theoretical results in modern computer science. Shor's algorithm (1994) demonstrated that a universal quantum computer could factor integers and compute discrete logarithms in polynomial time, threatening the foundations of RSA and Elliptic Curve Cryptography (ECC). Grover's algorithm (1996) established a quadratic speedup for unstructured search, effectively halving the security margin of symmetric ciphers. More recently, the Quantum Approximate Optimization Algorithm (QAOA) has emerged as a leading candidate for solving combinatorial optimization problems on near-term quantum hardware, with direct applications to lattice-based cryptanalysis.

However, the growing public interest in quantum computing has also produced a proliferation of *quantum-washing* — the presentation of classical heuristics in quantum terminology without genuine quantum mechanical content. This document provides a rigorous alternative: a complete, executable suite of actual quantum algorithms running on an exact state-vector simulator, with explicit architectural documentation and experimental validation. All code is provided under the Apache 2.0 license for educational and research use.

§02 System Architecture

State-Vector Simulator

QCS

The computational backend is a lightweight, gate-level quantum circuit simulator that stores the full 2^n complex amplitude vector and applies unitary operations via direct matrix-vector multiplication. The simulator supports single-qubit gates (H, X, Y, Z, RX, RY, RZ), two-qubit gates (CNOT, CZ, controlled-phase, SWAP), and multi-qubit primitives including the Quantum Fourier Transform (QFT) and its inverse. On consumer hardware, the simulator comfortably handles up to approximately 16 qubits (65,536 amplitudes), beyond which memory requirements grow exponentially.

PYTHON

```python
class QuantumSimulator:
    def __init__(self, n_qubits: int):
        self.n = n_qubits
        self.N = 1 << n_qubits          # 2^n amplitudes
        self.state = np.zeros(self.N, dtype=complex)
        self.state[0] = 1.0              # |0...0⟩

    def h(self, q: int):               # Hadamard gate
        stride = 1 << q
        for base in range(0, self.N, stride << 1):
            for j in range(base, base + stride):
                a, b = self.state[j], self.state[j + stride]
                self.state[j] = (a + b) / np.sqrt(2)
                self.state[j + stride] = (a - b) / np.sqrt(2)
```

Algorithm-to-Hardware Mapping

The following table establishes rigorous correspondences between high-level algorithmic concepts and their low-level quantum circuit implementations within the simulator.

ALGORITHMIC CONCEPT	QUANTUM PRIMITIVE	MATHEMATICAL OBJECT
Optimization layers (p	QAOA depth / Grover	Repetitions of cost-mixer or

ALGORITHMIC CONCEPT	QUANTUM PRIMITIVE	MATHEMATICAL OBJECT
= 3–5)	iteration count	oracle-diffusion
Problem Hamiltonian	Ising interaction coefficients in H_C	Diagonal cost Hamiltonian terms
Solution quality metric	Expectation value $\langle\psi\|H_C\|\psi\rangle$	Energy eigenvalue (lower = better solution)
Solution space exploration	Mixer Hamiltonian evolution $e^{(-i\beta H_X)}$	RX rotations exploring the computational basis
Classical optimization	Variational parameter tuning	Classical outer-loop objective function
Convergence precision	Parameter update tolerance	Hill-climbing step size and iteration bound

§03 Quantum Algorithms

QAOA

Variational quantum optimization for lattice problems. Maps SVP and LWE to Ising Hamiltonians.

Shor's

Quantum period-finding via QFT. Factors toy RSA instances using modular exponentiation oracles.

Grover's

Amplitude amplification for unstructured search. Quadratic speedup on symmetric key recovery.

QAOA for Lattice Cryptography

The Quantum Approximate Optimization Algorithm (Farhi et al., 2014) is applied to two fundamental lattice problems: the Shortest Vector Problem (SVP) and Learning With Errors (LWE). Both problems are encoded as diagonal cost Hamiltonians where the ground state corresponds to the optimal cryptanalytic solution.

For **SVP-2D**, integer coefficients (s, t) are encoded in qubit registers spanning [−3, +3] in each dimension. The cost function computes the squared norm $\|s \cdot b_1 + t \cdot b_2\|^2$ with a heavy penalty for the zero vector. For **LWE**, secret variables $s_i \in \mathbb{Z}_q$ are encoded in $\lceil \log_2(q) \rceil$ qubits each, and the cost measures the squared distance between A·s and the target vector b modulo q.

PYTHON

```python
def _apply_cost(self, gamma: float):
    # Apply e^(-i * gamma * H_C) where H_C is diagonal
    phases = np.exp(-1j * gamma * self._cost_array)
    self.sim.state *= phases
```

QCS

```
_circuit(self, params: np.ndarray):
self.sim.reset()
    self.sim.h_all()                    # Uniform superposition
    for layer in range(self.p):
        self._apply_cost(gammas[layer]) # Cost evolution
        for q in range(self.n):
            self.sim.rx(2 * betas[layer], q)  # Mixer evolution
```

Shor's Algorithm for Integer Factorization

Shor's algorithm (Shor, 1994) reduces integer factorization to order-finding via quantum period detection. The implementation follows the standard three-register architecture: a work register initialized in uniform superposition, an auxiliary register initialized to |1⟩, and a modular exponentiation oracle that computes a^x mod N reversibly. The Quantum Fourier Transform on the work register extracts the period r, from which factors are recovered via gcd($a^{r/2}$ ± 1, N).

PYTHON

```python
# Shor's core: period finding via QFT
for q in range(n_work):
    sim.h(q)                            # Superposition over x
sim.x(n_work)                           # Aux = |1⟩

for i in range(n_work):
    ap = pow(a, 2**i, N)
    _controlled_mult(sim, i, ap)        # |x⟩|1⟩ → |x⟩|a^x mod N⟩

sim.qft(list(range(n_work)))           # Extract period
```

Grover's Algorithm for Symmetric Key Search

Grover's algorithm (Grover, 1996) provides a quadratic speedup for unstructured search problems. For an *n*-bit key space, the optimal number of iterations is ⌊π/4 ·

$\sqrt{N}\rfloor$, where $N = 2^n$. The algorithm alternates between a phase oracle (which marks the correct key by flipping its phase) and a diffusion operator (which amplifies the amplitude of marked states).

Two toy ciphers are provided: a 4-bit XOR+S-box cipher and a 6-bit 2-round Feistel network. In both cases, Grover recovers the key with probability approaching the theoretical maximum (~96% for 4 bits with optimal iterations).

§04 Experimental Results

QAOA on 2D Shortest Vector Problem

Lattice basis: $\mathbf{b_1 = (3, 0)}$, $\mathbf{b_2 = (1, 2)}$. Search radius: ±3 in each dimension (6 qubits, p = 3). Classical brute-force optimum: s = 0, t = −1, norm² = 5.0.

SAMPLED (S, T)	VECTOR	SQUARED NORM	MEASUREMENT COUNT	STATUS
(−1, −1)	(−4.0, −2.0)	20.00	519 / 5000	Near-optimal
(−1, 1)	(−2.0, 2.0)	8.00	472 / 5000	Near-optimal
(0, −2)	(−2.0, −4.0)	20.00	454 / 5000	Near-optimal
(0, −1)	**(−1.0, −2.0)**	**5.00**	**Classical optimum**	Optimal

Shor's Algorithm — Factorization Results

COMPOSITE N	BASE A	PERIOD R	FACTORS	QUBITS USED	STATUS
15	4	2	3 × 5	10	Success
21	13	6	3 × 7	10	Success
35	4	6	5 × 7	12	Success

Grover's Algorithm — Key Recovery

Target: 4-bit XOR+S-box cipher. Key = `0110`. Optimal iterations: 2 ($\pi/4 \cdot \sqrt{16} \approx 3.14$, rounded to 2 for small N). Measurement shots: 2048.

MEASURED KEY	COUNT	PROBABILITY	STATUS
0110	**1975**	**96.4%**	Correct Key
0111	9	0.4%	Incorrect
1111	8	0.4%	Incorrect
1101	7	0.3%	Incorrect

Simulatable vs. Real-World Scale

TARGET	THIS SUITE (SIMULATED)	REAL-WORLD INSTANCE	SCALE GAP
SVP	2D, 6 qubits, ±3 radius	500+ dimensions (Kyber/Dilithium)	Exponential

TARGET	THIS SUITE (SIMULATED)	REAL-WORLD INSTANCE	SCALE GAP
LWE	n = 2, q = 4	Kyber: n = 256, q = 3329	Astronomical
RSA	N = 15–35 (4–6 bit primes)	RSA-2048 (1024-bit primes)	$\sim 2^{2000}\times$
AES	4-bit keys	AES-128	$\sim 2^{124}\times$

§05 References & Citations

[1] **P. W. Shor,** "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, Santa Fe, NM, USA, 1994, pp. 124–134. doi: 10.1109/SFCS.1994.365700

[2] **L. K. Grover,** "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, Philadelphia, PA, USA, 1996, pp. 212–219. doi: 10.1145/237814.237866

[3] **E. Farhi, J. Goldstone, and S. Gutmann,** "A Quantum Approximate Optimization Algorithm," *arXiv preprint arXiv:1411.4028*, Nov. 2014. Available: https://arxiv.org/abs/1411.4028

QCS

O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Journal of the ACM (JACM)*, vol. 56, no. 6, pp. 1–40, Sep. 2009. doi: 10.1145/1568318.1568324

[5] **NIST,** "Post-Quantum Cryptography Standardization," National Institute of Standards and Technology, 2022–2024. Available: https://csrc.nist.gov/projects/post-quantum-cryptography

[6] **M. A. Nielsen and I. L. Chuang,** *Quantum Computation and Quantum Information: 10th Anniversary Edition*, Cambridge University Press, 2010. ISBN: 978-1107002173

[7] **J. Preskill,** "Quantum Computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, Aug. 2018. doi: 10.22331/q-2018-08-06-79

[8] **G. E. N. Mc Kenzie,** "Quantum Cryptanalysis Suite: Simulated Qubits Doing Real Work," *Zenodo*, May 2026. doi: 10.5281/zenodo.19961317

§06 License

📜 Apache License, Version 2.0

Copyright © 2026 **Gerald Enrique Nelson Mc Kenzie**
ORCID: 0009-0008-6017-4922 | DOI: 10.5281/zenodo.19961317

```
Licensed under the Apache License, Version 2.0 (the "License");
```

QCS

```
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License.

DISCLAIMER: This software is provided for educational and research purposes
only. It implements quantum algorithms on classical simulators with severe
qubit limitations (≤ 16 qubits). It cannot and should not be used for actual
cryptanalysis of real-world cryptographic systems.
```

GitHub Repository Zenodo DOI ORCID Profile

Quantum Cryptanalysis Suite v1.0 | Published May 1, 2026
Author: Gerald Enrique Nelson Mc Kenzie | Licensed under Apache 2.0